

Code Sample

Long Version

Anonymous

This document contains excerpts from the replication code of a paper titled “Inequality in a Potential Equalizer of Opportunity: Variation in the Accessibility and Net Benefits of English-Medium Instruction in India”. Selected outputs are rendered at the end. To generate this PDF, run `make samples` or `scripts/run_full_build.sh` from the repo root.

Survey-weighted enrollment model

File: R/selection/estimate_selection_probit.R

```
selection_probit_variables <- function(selection_data) {
  controls <- c("AGE", "age", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "SECTOR")
  exclusion_all_kids <- c("DIST_FROM_NEAREST_PRIMARY_CLASS", "father_educ")
  exclusion_district <- c(
    "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",
    "dmean_num_RECD_SCHOLARSHIP_STIPEND", "dmean_num_RECD_TXT_BOOKS",
    "dmean_num_RECD_STATIONERY", "dmean_num_MID_DAY_MEAL_ETC_RECD",
    "dmean_num_ENROLLMENT_COST"
  )
  intersect(c(controls, exclusion_all_kids, exclusion_district), names(selection_data))
}

#' project the selection sample to the model's actual dependency surface
#'
#' Keeping unrelated Block-5 schooling attributes in the fitted survey design makes
#' the serialized model change whenever those attributes change, even though they
#' are absent from the probit formula. A narrow model frame lets {targets} skip the
#' expensive AME branch when upstream changes do not affect estimation.
project_selection_model_data <- function(selection_data) {
```

```

selection_data <- safe_df(selection_data)

design <- c("enrolled", "FSU_SL_NO", "weight", "STATE", "STRATUM", "SUB_STRATUM_NO")

keep <- unique(c(design, selection_probit_variables(selection_data)))

selection_data[intersect(keep, names(selection_data))]
}

#' estimate selection probit
#'
estimate_selection_probit <- function(selection_data, cfg) {
  if (!"enrolled" %in% names(selection_data) || all(is.na(selection_data$enrolled))) {
    return(list(status = "out_of_active_pipeline", reason = "No enrolled variable."))
  }

  covars <- selection_probit_variables(selection_data)
  if (!length(covars)) {
    return(list(status = "out_of_active_pipeline", reason = "No probit covariates."))
  }

  f_probit <- stats::reformulate(covars, response = "enrolled")
  if (identical(cfg$mode, "final") && requireNamespace("survey", quietly = TRUE)) {
    old_options <- options(survey.lonely.psu = "average")
    on.exit(options(old_options), add = TRUE)

    design <- build_survey_design_selection(selection_data)
    if (!is.null(design)) {
      out <- fit_selection_probit(design, f_probit)
      return(stabilize_selection_model_formula(out, f_probit))
    }
  }

  out <- stats::glm(f_probit, data = selection_data, family = stats::binomial(link = "probit"))
  stabilize_selection_model_formula(out, f_probit)
}

#' store a durable selection-model formula
#'
#' Programmatically fitted models otherwise retain a call to the local symbol
#' `f_probit`. Packages which reconstruct model data from the saved call cannot
#' resolve that symbol after the model is serialized by targets. Embed the
#' formula object in the call and retain the existing audit attribute.
stabilize_selection_model_formula <- function(model, formula) {
  if (!inherits(formula, "formula")) {
    stop("Selection model formula must inherit from formula.", call. = FALSE)
  }
}

```

```

if (!is.null(model$call)) model$call$formula <- formula
attr(model, "selection_probit_formula") <- formula
model
}

#' build survey design selection
#'
build_survey_design_selection <- function(selection_df) {
  psu <- first_col(selection_df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
  weight <- first_col(selection_df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
  strata_cols <- intersect(c("STATE", "STRATUM", "SUB_STRATUM_NO"), names(selection_df))
  if (is.null(psu) || is.null(weight) || !length(strata_cols)) return(NULL)
  selection_df$survey_strata <- interaction(selection_df[strata_cols], drop = TRUE)
  survey::svydesign(
    ids = stats::as.formula(paste0("~", psu)),
    strata = ~.survey_strata,
    weights = stats::as.formula(paste0("~", weight)),
    data = selection_df,
    nest = TRUE
  )
}

#' fit selection probit
#'
fit_selection_probit <- function(selection_design, f_probit) {
  survey::svyglm(f_probit, design = selection_design, family = stats::quasibinomial(link = "probit"))
}

```

Average marginal effects on the model estimation sample

File: R/selection/compute_average_marginal_effects.R

```

ame_model_weight <- function(model_data, model_weights = NULL) {
  weight_cols <- intersect(c("weight", "WEIGHT", "Multiplier", "multiplier", "(weights)"), names(model_data))
  if (length(weight_cols)) {
    return(suppressWarnings(as.numeric(model_data[[weight_cols[[1]]]])))
  }
}

```

```

if (!is.null(model_weights) && length(model_weights) == nrow(model_data)) {
  return(suppressWarnings(as.numeric(model_weights)))
}

NULL
}

#' build marginaleffects newdata and weights
#'
#' @return List with `data`, `wts`, and `has_explicit_weights`.
ame_model_data_and_weights <- function(model) {
  model_data <- as.data.frame(stats::model.frame(model))
  model_weights <- tryCatch(stats::weights(model), error = function(e) NULL)
  weight <- ame_model_weight(model_data, model_weights)

  if (!is.null(weight) && length(weight) == nrow(model_data) && any(is.finite(weight) & weight != 1)) {
    model_data$.ame_weight <- weight
    return(list(data = model_data, wts = ".ame_weight", has_explicit_weights = TRUE))
  }

  list(data = model_data, wts = FALSE, has_explicit_weights = FALSE)
}

#' run marginaleffects while muffling only a redundant explicit-weight warning
#'
run_avg_slopes <- function(model, model_data, wts, numberiv = NULL) {
  args <- list(
    model = model,
    newdata = model_data,
    wts = wts,
    vcov = TRUE,
    type = "response"
  )
  if (!is.null(numberiv)) args$numberiv <- numberiv

  withCallingHandlers(
    do.call(marginaleffects::avg_slopes, args),
    warning = function(w) {
      msg <- conditionMessage(w)
      explicit_weight_warning <- grepl(
        "normally good practice to specify weights using the `wts` argument",

```

```

      msg,
      fixed = TRUE
    )
    if (explicit_weight_warning && !identical(wts, FALSE)) {
      invokeRestart("muffleWarning")
    }
  }
)
}

#' compute ames autodiff
#'
compute_ames_autodiff <- function(model) {
  # Use the exact estimation sample retained by glm/svyglm. Passing the full
  # pre-model selection_data can have extra rows omitted during model fitting,
  # which makes marginaleffects recycle weights/covariates silently.
  amed <- ame_model_data_and_weights(model)
  run_avg_slopes(model, amed$data, amed$wts)
}

```

Montiel Olea–Pflueger effective-F calculation

File: R/iv/weak_identification.R

```

mop_effective_f <- function(
  model, analysis_data, tau = 0.10, size = 0.05) {
  unavailable <- function(reason) {
    list(
      statistic = NA_real_, critical_value = NA_real_, p.value = NA_real_,
      effective_df = NA_real_, tau = tau, size = size,
      status = "not_estimated", reason = reason
    )
  }

  if (!inherits(model, "ivreg")) {
    return(unavailable("Montiel Olea-Pflueger effective F requires an ivreg model."))
  }

  if (!requireNamespace("momentfit", quietly = TRUE)) {
    return(unavailable("Package 'momentfit' is not installed."))
  }
}

```

```

if (!is.finite(tau) || tau <= 0 || !is.finite(size) || size <= 0 || size >= 1) {
  return(unavailable("Montiel Olea-Pflueger tau and size must be valid probabilities."))
}

terms <- parse_iv_formula_terms(model)
if (is.null(terms)) {
  return(unavailable("Could not parse IV formula for Montiel Olea-Pflueger effective F."))
}

endogenous <- setdiff(terms$regressors, terms$instruments)
if (length(endogenous) != 1L) {
  return(unavailable("Montiel Olea-Pflueger effective F requires exactly one endogenous regressor."))
}

formula <- stats::formula(model)
regression_formula <- stats::as.formula(
  call("~", formula[[2L]], formula[[3L]][[2L]]),
  env = environment(formula)
)
instrument_formula <- stats::as.formula(
  call("~", formula[[3L]][[3L]]),
  env = environment(formula)
)
needed <- unique(c(all.vars(regression_formula), all.vars(instrument_formula)))
data <- iv_analysis_frame(analysis_data, needed)
missing <- setdiff(needed, names(data))
if (length(missing)) {
  return(unavailable(paste(
    "Missing variables for Montiel Olea-Pflueger effective F:",
    paste(missing, collapse = ", ")
  )))
}
moment_data <- data[needed]
if (any(!stats::complete.cases(moment_data))) {
  return(unavailable("Montiel Olea-Pflueger analysis data contain incomplete model rows."))
}

stored_cluster <- attr(model, "cluster_state", exact = TRUE)
cluster <- if (
  length(stored_cluster) == stats::nobs(model) &&
  !anyNA(stored_cluster) && length(unique(stored_cluster)) >= 2L
) {

```

```

    as.vector(stored_cluster)
  } else {
    iv_model_cluster(model, data)
  }
covariance <- if (is.null(cluster)) "MDS" else "CL"
covariance_options <- if (is.null(cluster)) {
  list(type = "HCO")
} else {
  # momentfit's clustered moment covariance is based on sandwich::meatCL()
  # and supports HCO. Keep its package-native finite-cluster correction
  # rather than altering the existing HC1 Wald-F diagnostic to force equality.
  list(
    cluster = data.frame(cluster = cluster),
    type = "HCO", cadjust = TRUE, multi0 = FALSE
  )
}

result <- tryCatch({
  moment_model <- momentfit::momentModel(
    regression_formula,
    instrument_formula,
    data = moment_data,
    vcov = covariance,
    vcovOptions = covariance_options
  )
  momentfit::MOPtest(
    moment_model,
    tau = tau,
    size = size,
    estMethod = "TSLS",
    simplified = TRUE,
    print = FALSE
  )
}, error = function(e) e)
if (inherits(result, "error")) return(unavailable(conditionMessage(result)))

values <- suppressWarnings(as.numeric(result[c("Feff", "critValue", "pvalue", "Keff"))])
names(values) <- c("statistic", "critical_value", "p.value", "effective_df")
if (length(values) != 4L || any(!is.finite(values))) {
  return(unavailable("momentfit::MOPtest() did not return finite effective-F diagnostics."))
}

```

```

c(
  as.list(values),
  list(tau = tau, size = size, status = "estimated", reason = NA_character_)
)
}

```

Anderson–Rubin inference and bounded exclusion sensitivity

File: R/iv/weak_identification.R

```

anderson_rubin_test <- function(
  data, outcome, treatment, excluded, included = character(),
  controls = character(), fixed_effect = "none", cluster, beta0 = 0
) {
  transformed <- ".ar_outcome"
  data[[transformed]] <- num(data[[outcome]]) - beta0 * num(data[[treatment]])
  rhs <- unique(c(excluded, included, controls, iv_fixed_effect_terms(fixed_effect)))
  fit <- stats::lm(stats::reformulate(rhs, response = transformed), data = data)
  test <- clustered_joint_wald_test(fit, excluded, cluster)
  c(statistic = test[["statistic"]], p.value = test[["p.value"]])
}

# Weak-IV-robust exclusion sensitivity for one scalar excluded instrument.
# For a candidate structural effect beta, regress Y - beta D on Z and the same
# nuisance terms used by the registered IV specification. The coefficient on Z
# is the direct effect gamma that would reconcile that beta with the data.
# Subtracting any fixed gamma * Z changes that coefficient but not the residuals,
# so the clustered standard error is invariant to gamma. This makes the union of
# Anderson--Rubin confidence sets over gamma in [lower, upper] available without
# a numerical gamma grid.

bounded_exclusion_ar_profile <- function(
  data, outcome, treatment, excluded, included = character(),
  controls = character(), fixed_effect = "none", cluster, beta_values) {
  excluded <- plain_chr(excluded)
  if (length(excluded) != 1L || is.na(excluded[[1L]]) || !nzchar(excluded[[1L]])) {
    stop("Bounded exclusion sensitivity requires exactly one excluded instrument.", call. = FALSE)
  }
  beta_values <- num(beta_values)
  if (!length(beta_values) || any(!is.finite(beta_values))) {

```

```

    stop("Bounded exclusion sensitivity requires finite beta values.", call. = FALSE)
  }
  instrument <- excluded[[1L]]
  rhs <- unique(c(instrument, included, controls, iv_fixed_effect_terms(fixed_effect)))

  safe_bind_rows(lapply(beta_values, function(beta0) {
    transformed <- ".bounded_exclusion_ar_outcome"
    x <- as.data.frame(data)
    x[[transformed]] <- num(x[[outcome]]) - beta0 * num(x[[treatment]])
    fit <- stats::lm(stats::reformulate(rhs, response = transformed), data = x)
    inference <- iv_clustered_inference(fit, cluster)
    term <- model_term_inference(fit, instrument, inference$vcov)
    data.frame(
      beta = beta0,
      direct_effect_estimate = term[["estimate"]],
      direct_effect_std.error = term[["std.error"]],
      residual_df = stats::df.residual(fit),
      status = if (
        identical(inference$status, "estimated") &&
        all(is.finite(term[c("estimate", "std.error")])) &&
        term[["std.error"]] > 0
      ) "estimated" else "inference_unavailable",
      reason = if (identical(inference$status, "unavailable")) inference$reason else NA_character_,
      stringsAsFactors = FALSE
    )
  }))
}

```

Linked state-sector price series

File: R/prices/build_temporal_price_series.R

```

build_temporal_price_series <- function(
  price_sources,
  switch_date = as.Date("2013-01-01"),
  overlap_start = as.Date("2013-01-01"),
  overlap_end = as.Date("2014-12-01"),
  minimum_link_months = 6L,
  pre_switch_start = NULL,
  pre_switch_end = NULL) {

```

```

switch_date <- price_boundary(switch_date)
old <- select_pre_2013_price_series(price_sources)
new <- select_post_2013_price_series(price_sources)
links <- summarise_price_links(old, new, overlap_start, overlap_end)

minimum_link_months <- as.integer(minimum_link_months)
if (length(minimum_link_months) != 1L || is.na(minimum_link_months) || minimum_link_months < 1L) {
  stop("minimum_link_months must be one positive integer.", call. = FALSE)
}

old_groups <- unique(old[old$period < switch_date, c("state_code", "sector"), drop = FALSE])
linked_groups <- links[links$link_months >= minimum_link_months, c("state_code", "sector"), drop = FALSE]
linked_key <- paste(linked_groups$state_code, linked_groups$sector, sep = "\r")
missing_key <- !paste(old_groups$state_code, old_groups$sector, sep = "\r") %in% linked_key
if (any(missing_key)) {
  bad <- old_groups[missing_key, , drop = FALSE]
  stop(
    "Pre-2013 price series lack a sufficient direct link to CPI-R/U: ",
    paste(paste(bad$state_code, bad$sector, sep = "/"), collapse = ", "),
    call. = FALSE
  )
}

old <- merge(old, links, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
old <- old[old$period < switch_date, , drop = FALSE]
if (!is.null(pre_switch_start) || !is.null(pre_switch_end)) {
  if (is.null(pre_switch_start) || is.null(pre_switch_end)) {
    stop("Both pre_switch_start and pre_switch_end are required when trimming the old price series.",
      call. = FALSE)
  }
  pre_switch_start <- price_boundary(pre_switch_start)
  pre_switch_end <- price_boundary(pre_switch_end)
  if (pre_switch_end < pre_switch_start || pre_switch_end >= switch_date) {
    stop("The pre-switch production window must end before the switch date.", call. = FALSE)
  }
  old <- old[old$period >= pre_switch_start & old$period <= pre_switch_end, , drop = FALSE]
}
old$index_unlinked <- old$index
old$index <- num(old$index) * num(old$link_factor)

new <- new[new$period >= switch_date, , drop = FALSE]

```

```

new$link_factor <- 1
new$link_months <- NA_integer_
new$link_ratio_mad <- NA_real_
new$link_ratio_min <- NA_real_
new$link_ratio_max <- NA_real_
new$index_unlinked <- new$index

columns <- c(
  "state_code", "sector", "period", "index", "index_unlinked", "price_source",
  "link_factor", "link_months", "link_ratio_mad", "link_ratio_min", "link_ratio_max"
)
index <- rbind(old[columns], new[columns])
index$year <- as.integer(format(index$period, "%Y"))
index$month <- as.integer(format(index$period, "%m"))
index <- index[order(index$state_code, index$sector, index$period), , drop = FALSE]
rownames(index) <- NULL
validate_temporal_price_chain(index, switch_date)

structure(
  list(index = index, links = links, switch_date = switch_date),
  class = "emi_temporal_price_series"
)
}

```

State-sector real-consumption deflator construction

File: R/prices/price_deflators.R

```

build_state_sector_deflator <- function(
  temporal_index, spatial_relatives, reference_period = NULL, reference_index = NULL) {
  idx <- safe_df(temporal_index)
  spatial <- safe_df(spatial_relatives)
  validate_price_index(idx, keys = c("state_code", "sector", "period"))
  required_spatial <- c("state_code", "sector", "spatial_price_relative")
  missing <- setdiff(required_spatial, names(spatial))
  if (length(missing)) {
    stop("Spatial price relatives are missing columns: ", paste(missing, collapse = ", "), call. = FALSE)
  }
  if (anyDuplicated(spatial[c("state_code", "sector")])) {
    stop("Spatial price relatives must have one row per state and sector.", call. = FALSE)
  }
}

```

```

}

if (any(!positive_finite(num(spatial$spatial_price_relative)))) {
  stop("Spatial price relatives must be positive and finite.", call. = FALSE)
}

if (is.null(reference_index)) {
  if (is.null(reference_period)) stop("A price reference period or reference-index table is required.",
    ↪ call. = FALSE)
  reference_period <- as.Date(reference_period)
  ref <- idx[idx$period %in% reference_period, , drop = FALSE]
  if (!nrow(ref)) stop("No observations fall in the requested price reference period.", call. = FALSE)
  ref_mean <- stats::aggregate(index ~ state_code + sector, ref, function(x) mean(num(x), na.rm = TRUE))
  names(ref_mean)[names(ref_mean) == "index"] <- "reference_index"
} else {
  ref_mean <- safe_df(reference_index)
  required_reference <- c("state_code", "sector", "reference_index")
  missing_reference <- setdiff(required_reference, names(ref_mean))
  if (length(missing_reference)) {
    stop("Reference-price table is missing columns: ", paste(missing_reference, collapse = ", "), call. =
      ↪ FALSE)
  }
  if (anyDuplicated(ref_mean[c("state_code", "sector")]) ||
    ↪ any(!positive_finite(ref_mean$reference_index))) {
    stop("Reference-price table must have one positive row per state-sector.", call. = FALSE)
  }
  ref_mean <- ref_mean[required_reference]
}

out <- merge(
  idx,
  spatial,
  by = c("state_code", "sector"), all.x = TRUE, sort = FALSE
)

out <- merge(out, ref_mean, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
if ("source_state_code" %in% names(out) && any(!positive_finite(out$reference_index))) {
  donor_reference <- ref_mean
  names(donor_reference)[names(donor_reference) == "state_code"] <- "source_state_code"
  names(donor_reference)[names(donor_reference) == "reference_index"] <- "donor_reference_index"
  out <- merge(out, donor_reference, by = c("source_state_code", "sector"), all.x = TRUE, sort = FALSE)
  missing_reference <- !positive_finite(out$reference_index)
  out$reference_index[missing_reference] <- out$donor_reference_index[missing_reference]
  out$donor_reference_index <- NULL
}

```

```

}
out$temporal_price_relative <- num(out$index) / num(out$reference_index)
out$price_deflator <- num(out$spatial_price_relative) * num(out$temporal_price_relative)
if (any(!positive_finite(out$price_deflator))) {
  stop("Deflator construction left missing or invalid values.", call. = FALSE)
}
out <- out[order(out$state_code, out$sector, out$period), , drop = FALSE]
rownames(out) <- NULL
out
}

build_state_sector_price_deflators <- function(
  temporal_series,
  state_rules = read_price_state_crosswalk(),
  poverty_lines = read_tendulkar_poverty_lines(),
  reference_period = seq(as.Date("2011-07-01"), as.Date("2012-06-01"), by = "month"),
  reference_index = NULL,
  start_period = NULL,
  end_period = NULL,
  reference_rupees = 816,
  require_complete_grid = TRUE) {
  temporal_index <- if (inherits(temporal_series, "emi_temporal_price_series")) {
    temporal_series$index
  } else {
    safe_df(temporal_series)
  }
  expanded <- apply_price_state_rules(
    temporal_index,
    state_rules,
    start_period = start_period,
    end_period = end_period,
    require_complete_grid = require_complete_grid
  )
  spatial <- build_tendulkar_spatial_relatives(poverty_lines, reference_rupees)
  missing_spatial <- setdiff(
    unique(paste(expanded$state_code, expanded$sector, sep = "\r")),
    unique(paste(spatial$state_code, spatial$sector, sep = "\r"))
  )
  if (length(missing_spatial)) {
    stop("Tendulkar metadata does not cover every temporal state-sector series.", call. = FALSE)
  }
}

```

```

build_state_sector_deflator(
  expanded,
  spatial,
  reference_period = reference_period,
  reference_index = reference_index
)
}

```

Cross-vintage district candidate scoring

File: R/districts/lineage_matching.R

```

score_match_candidates <- function(source_roster, reference_units, excluded_source_ids = character()) {
  need_pkg("stringdist", "district-lineage candidate scores")
  source_roster <- safe_df(source_roster)
  reference_units <- safe_df(reference_units)
  if (!"raw_district" %in% names(source_roster)) source_roster$raw_district <- source_roster$district_std
  source_open <- source_roster[!source_roster$source_row_id %in% excluded_source_ids, , drop = FALSE]
  if (!nrow(source_open)) return(empty_source_candidates())

  pairs <- merge(
    source_open, reference_units,
    by = "state_std", all = FALSE, sort = FALSE,
    suffixes = c("_source", "_candidate")
  )
  if (!nrow(pairs)) return(empty_source_candidates())
  source_name <- pairs$district_std_source
  candidate_name <- pairs$district_std_candidate
  pairs$jw <- 1 - stringdist::stringdist(source_name, candidate_name, method = "jw", p = 0.1)
  dl_distance <- stringdist::stringdist(source_name, candidate_name, method = "dl")
  denom <- pmax(nchar(source_name), nchar(candidate_name), 1L)
  pairs$dl <- pmax(0, 1 - dl_distance / denom)
  pairs$trigram <- stringdist::stringsim(source_name, candidate_name, method = "cosine", q = 3)
  pairs$token <- token_jaccard_similarity(source_name, candidate_name)
  pairs$score <- rowMeans(pairs[c("jw", "dl", "trigram", "token")], na.rm = TRUE)

  ranked <- safe_bind_rows(lapply(split(seq_len(nrow(pairs)), pairs$source_row_id), function(i) {
    x <- pairs[i, , drop = FALSE]
    preference <- vintage_preference(x$wave[[1]])
    vintage_rank <- match(x$reference_vintage, preference, nomatch = length(preference) + 1L)
  }

```

```

x <- x[order(-x$score, -x$jw, vintage_rank, x$unit_id), , drop = FALSE]
x$rank <- match(x$district_std_candidate, unique(x$district_std_candidate))
distinct_scores <- x$score[!duplicated(x$district_std_candidate)]
second <- if (length(distinct_scores) >= 2L) distinct_scores[[2]] else -Inf
x$margin <- ifelse(x$rank == 1L, x$score - second, NA_real_)
x$preferred_vintage <- !duplicated(x$district_std_candidate)
x
}))

candidate_groups <- split(seq_len(nrow(ranked)), ranked$unit_id)
best_by_candidate <- stats::setNames(
  vapply(candidate_groups, function(i) {
    score <- ranked$score[i]
    ranked$source_row_id[i[which.max(score)]]
  }, character(1)),
  names(candidate_groups)
)

ranked$reciprocal_nearest <- ranked$rank == 1L &
  unname(best_by_candidate[ranked$unit_id]) == ranked$source_row_id
threshold <- district_match_candidate_thresholds()
ranked$high_precision_candidate <- ranked$rank == 1L & ranked$preferred_vintage &
  ranked$reciprocal_nearest &
  directional_tokens_compatible(ranked$district_std_source, ranked$district_std_candidate) &
  ranked$jw >= threshold[["jw"]] & ranked$d1 >= threshold[["d1"]] &
  ranked$trigram >= threshold[["trigram"]] & ranked$margin >= threshold[["margin"]]
ranked <- ranked[ranked$rank <= 5L, , drop = FALSE]

data.frame(
  source_row_id = ranked$source_row_id,
  wave = ranked$wave,
  source_code = ranked$source_code,
  state_std = ranked$state_std,
  source_name_raw = ranked$raw_district,
  source_name = ranked$district_std_source,
  candidate_unit = ranked$unit_id,
  candidate_name = ranked$district_std_candidate,
  candidate_source_id = ranked$source_id,
  reference_vintage = ranked$reference_vintage,
  candidate_method = "fuzzy_name_candidate",
  rank = ranked$rank,
  jw = ranked$jw,

```

```

dl = ranked$dl,
trigram = ranked$trigram,
token = ranked$token,
score = ranked$score,
margin = ranked$margin,
reciprocal_nearest = ranked$reciprocal_nearest,
high_precision_candidate = ranked$high_precision_candidate,
stringsAsFactors = FALSE
)
}

```

Residual spatial-autocorrelation diagnostics

File: R/diagnostics/diagnose_spatial_autocorrelation.R

```

spatial_moran_test_from_model_residuals <- function(model, district_panel, weights, legacy_name, estimand,
↪ variable, source) {
  if (!inherits(model, "ivreg")) {
    return(spatial_autocorrelation_status_row("out_of_active_pipeline", paste0("No active IV model for ",
↪ estimand, ".")))
  }
  rows <- spatial_model_rows(model, district_panel)
  if (!identical(as.integer(rows), as.integer(weights$row_index))) {
    weights <- build_spatial_weights_for_rows(
      district_panel, rows, queen = identical(weights$contiguity, "queen")
    )
  }
  x <- tryCatch(stats::residuals(model), error = function(e) NA_real_)
  compute_moran_tests(x, weights, legacy_name, estimand, variable, source)
}

spatial_moran_test_from_first_stage_residuals <- function(model, district_panel, weights, legacy_name,
↪ estimand, variable, source) {
  fit <- spatial_first_stage_model(model, district_panel)
  if (is.null(fit)) {
    return(spatial_autocorrelation_status_row("out_of_active_pipeline", paste0("Could not estimate
↪ first-stage model for ", estimand, ".")))
  }
  rows <- spatial_model_rows(fit, district_panel)
  if (!identical(as.integer(rows), as.integer(weights$row_index))) {

```

```

    weights <- build_spatial_weights_for_rows(
      district_panel, rows, queen = identical(weights$contiguity, "queen")
    )
  }
  x <- tryCatch(stats::residuals(fit), error = function(e) NA_real_)
  compute_moran_tests(x, weights, legacy_name, estimand, variable, source)
}

spatial_first_stage_model <- function(model, district_panel) {
  if (!inherits(model, "ivreg")) return(NULL)
  iv_terms <- parse_iv_formula_terms(model)
  if (is.null(iv_terms) || !length(iv_terms$regressors) || !length(iv_terms$instruments)) return(NULL)
  endogenous <- setdiff(iv_terms$regressors, iv_terms$instruments)
  if (!length(endogenous)) endogenous <- iv_terms$regressors[[1]]
  if (!length(endogenous) || is.na(endogenous[[1]]) || !nzchar(endogenous[[1]])) return(NULL)
  first_stage_formula <- stats::as.formula(paste(endogenous[[1]], "~", paste(iv_terms$instruments, collapse =
↪ " + ")))
  data <- as.data.frame(district_panel)
  missing <- setdiff(all.vars(first_stage_formula), names(data))
  if (length(missing)) return(NULL)
  tryCatch(stats::lm(first_stage_formula, data = data), error = function(e) NULL)
}

#' compute moran tests
#'
#' @return A data frame containing the statistic, Moran's I estimate, expected
#' value, variance, and p-value. `moran.test()` is the asymptotic-normal test
#' used by the legacy Rmd.
compute_moran_tests <- function(x, spatial_weights, legacy_name = NA_character_, estimand = NA_character_,
↪ variable = NA_character_, source = NA_character_) {
  if (!inherits(spatial_weights, "emi_spatial_weights") || !identical(spatial_weights$status, "constructed"))
↪ {
    return(spatial_autocorrelation_status_row("out_of_active_pipeline", "Spatial weights were not
↪ constructed."))
  }
  x <- suppressWarnings(as.numeric(x))
  if (length(x) != length(spatial_weights$row_index)) {
    return(spatial_autocorrelation_status_row("out_of_active_pipeline", paste0("Length mismatch for ",
↪ variable, ": ", length(x), " values for ", length(spatial_weights$row_index), " spatial rows.")))
  }
  if (any(!is.finite(x))) {

```

```

    return(spatial_autocorrelation_status_row("out_of_active_pipeline", paste0("Non-finite values prevent
    ↪ Moran's I for ", variable, ".")))
  }

test <- tryCatch(spdep::moran.test(x, spatial_weights$listw, zero.policy = TRUE), error = function(e) e)
if (inherits(test, "error")) {
  return(spatial_autocorrelation_status_row("out_of_active_pipeline", conditionMessage(test)))
}

estimate <- suppressWarnings(as.numeric(test$estimate))
estimate_value <- function(i) if (length(estimate) >= i && is.finite(estimate[[i]])) estimate[[i]] else
↪ NA_real_

data.frame(
  legacy_name = legacy_name,
  estimand = estimand,
  variable = variable,
  source = source,
  test = "moran",
  status = "estimated",
  statistic = suppressWarnings(as.numeric(test$statistic[[1]])),
  estimate = estimate_value(1L),
  expected = estimate_value(2L),
  variance = estimate_value(3L),
  p.value = suppressWarnings(as.numeric(test$p.value)),
  method = test$method %||% "Moran I test under randomisation",
  alternative = test$alternative %||% NA_character_,
  n = length(x),
  contiguity = spatial_weights$contiguity %||% NA_character_,
  weights_style = spatial_weights$style %||% NA_character_,
  matrix_style = spatial_weights$matrix_style %||% NA_character_,
  zero_policy = spatial_weights$zero_policy %||% NA,
  n_spatial_rows = spatial_weights$n %||% length(x),
  n_islands = spatial_weights$n_islands %||% NA_integer_,
  mean_neighbors = spatial_weights$mean_neighbors %||% NA_real_,
  warnings = paste(spatial_weights$warnings %||% character(), collapse = "; "),
  reason = NA_character_,
  stringsAsFactors = FALSE
)
}

```

Selected Paper Outputs

These outputs are the same formatted table and figure files used by the paper; their presentation is assembled in `R/output/make_tables.R` and `R/output/make_figures.R`.

Table 1: Correlates of School Enrollment

Term	Enrolled (1 = yes)
Age (years)	-0.028*** (0.001)
Female (ref: Male)	0.044*** (0.009)
Household size	-0.004* (0.002)
Religion: Muslim (ref: Hindu)	-0.125*** (0.013)
Religion: Christian	-0.042 (0.037)
Religion: Sikh	0.040 (0.087)
Religion: Jain	-0.001 (0.037)
Religion: Buddhist	0.047 (0.066)
Religion: Zoroastrian	0.100 (0.157)
Religion: Other	0.250 (13.784)
Social group: Scheduled Tribe (ref: Other)	-0.101*** (0.019)
Social group: Scheduled Caste	-0.074*** (0.015)
Social group: Other Backward Class	-0.029** (0.012)
Urban (ref: Rural)	-0.003 (0.012)
Distance 1–2km (ref: <1km)	-0.076* (0.044)
Distance 2–3km	-0.001 (0.020)
Distance 3–5km	0.004 (0.184)
Distance > 5km	0.036 (0.104)
Male household educ.: Literate, no school (ref: Illiterate)	0.098** (0.043)
Male household educ.: Literate, school < primary	0.129*** (0.018)
Male household educ.: Primary	0.144*** (0.015)
Male household educ.: Upper primary	0.197*** (0.019)
Male household educ.: Secondary	0.215*** (0.027)
Male household educ.: Higher secondary	0.251*** (0.043)
Male household educ.: Postsecondary+	0.249***

Table 2: Weak-IV-Robust Long-Run Inference

	2022-23 long change	2023-24 long change
EMI exposure	0.117 (0.152)	0.137 (0.234)
MOP effective F	0.69	0.41
AR p -value at zero	0.001	0.014
AR 95% accepted set	$[-0.193, -0.071] \cup [0.025, 0.193]$	$[-0.186, -0.051] \cup [0.015, 0.186]$
Direct effect needed to admit zero	41.7%	19.9%
Observations	524	522

The table reports conventional 2SLS coefficients; standard errors are state-clustered. Weak-identification assessment uses the Montiel Olea–Pflueger effective F statistic and Anderson–Rubin inference. The 95% AR accepted sets are displayed as unions of intervals on the evaluated inversion grid, and both reported sets reach the grid boundaries. The direct-effect row gives the smallest permitted direct effect, as a share of the absolute reduced form, for which the AR set includes $\beta = 0$. Sources: Census of India 2001; NSS 61st Round, 2004–05; NSS 64th Round, 2007–08; HCES 2022–23; HCES 2023–24.

Table 1: Correlates of School Enrollment (*continued*)

Term	Enrolled (1 = yes)
	(0.044)

* $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$. Entries are average changes in predicted enrollment probability. Continuous covariates are average slopes; categorical covariates are average discrete comparisons against the stated reference category, evaluated over the observed covariate distribution. Standard errors use the declared NSS survey design, including household weights, primary sampling units, and strata formed from state, stratum, and substratum identifiers. The fitted model includes district averages of schooling costs and benefits among enrolled children. Their coefficients are omitted because those variables combine local provision with the composition of enrolled children. Source: NSS 64th Round, 2007–08, Participation and Expenditure in Education.

Consumption-series consistency

Validation of the harmonized consumption series used in the later-outcome analysis.

